

目 录

致谢

1. 快速开始

2. 数据库

3. 模型

4. CRUD:读写数据

5. 关联

6. Callbacks

7. 高级用法

8. 开发

9. 更新日志

10. 译者日志

致谢

当前文档《GORM 中文文档》由 进击的皇虫 使用 书栈 (BookStack.CN) 进行构建, 生成于 2018-05-17。

书栈(BookStack.CN) 仅提供文档编写、整理、归类等功能, 以及对文档内容的生成和导出工具。

文档内容由网友们编写和整理, 书栈(BookStack.CN) 难以确认文档内容知识点是否错漏。如果您在阅读文档获取知识的时候, 发现文档内容有不恰当的地方, 请向我们反馈, 让我们共同携手, 将知识准确、高效且有效地传递给每一个人。

同时, 如果您在日常生活、工作和学习中遇到有价值有营养的知识文档, 欢迎分享到 书栈(BookStack.CN) , 为知识的传承献上您的一份力量!

如果当前文档生成时间太久, 请到 书栈(BookStack.CN) 获取最新的文档, 以跟上知识更新换代的步伐。

文档地址: <http://www.bookstack.cn/books/gorm-cn-doc>

书栈官网: <http://www.bookstack.cn>

书栈开源: <https://github.com/TruthHun>

分享, 让知识传承更久远! 感谢知识的创造者, 感谢知识的分享者, 也感谢每一位阅读到此处的读者, 因为我们都将成为知识的传承者。

1. 快速开始

- [GORM 中文文档](#)
 - [概述](#)
 - [安装](#)
 - [升级到V1.0](#)
 - [快速开始](#)
 - [来源\(书栈小编注\)](#)

GORM 中文文档

<http://gorm.book.jasperxu.com/>

Golang写的，开发人员友好的ORM库。

概述

- 全功能ORM (几乎)
- 关联 (包含一个, 包含多个, 属于, 多对多, 多种包含)
- Callbacks (创建/保存/更新/删除/查找之前/之后)
- 预加载 (急加载)
- 事务
- 复合主键
- SQL Builder
- 自动迁移
- 日志
- 可扩展, 编写基于GORM回调的插件
- 每个功能都有测试
- 开发人员友好

安装

```
1. go get -u github.com/jinzhu/gorm
```

升级到V1.0

- [更新日志](#)

快速开始

```
1. package main
2.
3. import (
4.     "github.com/jinzhu/gorm"
5.     _ "github.com/jinzhu/gorm/dialects/sqlite"
6. )
7.
8. type Product struct {
9.     gorm.Model
10.    Code string
11.    Price uint
12. }
13.
14. func main() {
15.    db, err := gorm.Open("sqlite3", "test.db")
16.    if err != nil {
17.        panic("连接数据库失败")
18.    }
19.    defer db.Close()
20.
21.    // 自动迁移模式
22.    db.AutoMigrate(&Product{})
23.
24.    // 创建
25.    db.Create(&Product{Code: "L1212", Price: 1000})
```

```
26.  
27.    // 读取  
28.    var product Product  
29.    db.First(&product, 1) // 查询id为1的product  
30.    db.First(&product, "code = ?", "L1212") // 查询code为L1212的  
        product  
31.  
32.    // 更新 - 更新product的price为2000  
33.    db.Model(&product).Update("Price", 2000)  
34.  
35.    // 删除 - 删除product  
36.    db.Delete(&product)  
37. }
```

来源(书栈小编注)

<https://github.com/jasperxu/gorm-cn-doc>

2. 数据库

- 数据库
 - 连接数据库 {#dbc}
 - MySQL
 - PostgreSQL
 - Sqlite3
 - 不支持的数据库
 - 迁移 {#m}
 - 自动迁移
 - 检查表是否存在
 - 创建表
 - 删除表
 - 修改列
 - 删除列
 - 添加外键
 - 索引

数据库

连接数据库 {#dbc}

要连接到数据库首先要导入驱动程序。例如

```
1. import _ "github.com/go-sql-driver/mysql"
```

为了方便记住导入路径，GORM包装了一些驱动。

```
1. import _ "github.com/jinzhu/gorm/dialects/mysql"
2. // import _ "github.com/jinzhu/gorm/dialects/postgres"
```

```
3. // import _ "github.com/jinzhu/gorm/dialects/sqlite"
4. // import _ "github.com/jinzhu/gorm/dialects/mssql"
```

MySQL

注：为了处理 `time.Time` ，您需要包括 `parseTime` 作为参数。（[更多支持的参数](#)）

```
1. import (
2.     "github.com/jinzhu/gorm"
3.     _ "github.com/jinzhu/gorm/dialects/mysql"
4. )
5.
6. func main() {
7.     db, err := gorm.Open("mysql", "user:password@/dbname?
        charset=utf8&parseTime=True&loc=Local")
8.     defer db.Close()
9. }
```

PostgreSQL

```
1. import (
2.     "github.com/jinzhu/gorm"
3.     _ "github.com/jinzhu/gorm/dialects/postgres"
4. )
5.
6. func main() {
7.     db, err := gorm.Open("postgres", "host=myhost user=gorm
        dbname=gorm sslmode=disable password=mypassword")
8.     defer db.Close()
9. }
```

Sqlite3

```
1. import (
```

```

2.     "github.com/jinzhu/gorm"
3.     _ "github.com/jinzhu/gorm/dialects/sqlite"
4. )
5.
6. func main() {
7.     db, err := gorm.Open("sqlite3", "/tmp/gorm.db")
8.     defer db.Close()
9. }

```

不支持的数据库

GORM正式支持上述的数据库，如果您使用的是不受支持的数据库请按照下面的连接编写对应数据库支持文件。

<https://github.com/jinzhu/gorm/blob/master/dialect.go>

迁移 {#m}

自动迁移

自动迁移模式将保持更新到最新。

警告：自动迁移仅仅会创建表，缺少列和索引，并且不会改变现有列的类型或删除未使用的列以保护数据。

```

1. db.AutoMigrate(&User{})
2.
3. db.AutoMigrate(&User{}, &Product{}, &Order{})
4.
5. // 创建表时添加表后缀
6. db.Set("gorm:table_options", "ENGINE=InnoDB").AutoMigrate(&User{})

```

检查表是否存在

```

1. // 检查模型`User`表是否存在
2. db.HasTable(&User{})
3.
4. // 检查表`users`是否存在
5. db.HasTable("users")

```

创建表

```

1. // 为模型`User`创建表
2. db.CreateTable(&User{})
3.
4. // 创建表`users`时将“ENGINE = InnoDB”附加到SQL语句
5. db.Set("gorm:table_options", "ENGINE=InnoDB").CreateTable(&User{})

```

删除表

```

1. // 删除模型`User`的表
2. db.DropTable(&User{})
3.
4. // 删除表`users`
5. db.DropTable("users")
6.
7. // 删除模型`User`的表和表`products`
8. db.DropTableIfExists(&User{}, "products")

```

修改列

修改列的类型为给定值

```

1. // 修改模型`User`的description列的数据类型为`text`
2. db.Model(&User{}).ModifyColumn("description", "text")

```

删除列

```
1. // 删除模型`User`的description列
2. db.Model(&User{}).DropColumn("description")
```

添加外键

```
1. // 添加主键
2. // 1st param : 外键字段
3. // 2nd param : 外键表(字段)
4. // 3rd param : ONDELETE
5. // 4th param : ONUPDATE
6. db.Model(&User{}).AddForeignKey("city_id", "cities(id)",
    "RESTRICT", "RESTRICT")
```

索引

```
1. // 为`name`列添加索引`idx_user_name`
2. db.Model(&User{}).AddIndex("idx_user_name", "name")
3.
4. // 为`name`, `age`列添加索引`idx_user_name_age`
5. db.Model(&User{}).AddIndex("idx_user_name_age", "name", "age")
6.
7. // 添加唯一索引
8. db.Model(&User{}).AddUniqueIndex("idx_user_name", "name")
9.
10. // 为多列添加唯一索引
11. db.Model(&User{}).AddUniqueIndex("idx_user_name_age", "name",
    "age")
12.
13. // 删除索引
14. db.Model(&User{}).RemoveIndex("idx_user_name")
```

3. 模型

- 模型

- 模型定义 {#md}
- 约定 {#c}
 - `gorm.Model` 结构体
 - 表名是结构体名称的复数形式
 - 更改默认表名
 - 列名是字段名的蛇形小写
 - 字段 `ID` 为主键
 - 字段 `CreatedAt` 用于存储记录的创建时间
 - 字段 `UpdatedAt` 用于存储记录的修改时间
 - 字段 `DeletedAt` 用于存储记录的删除时间，如果字段存在

模型

模型定义 {#md}

```

1. type User struct {
2.     gorm.Model
3.     Birthday    time.Time
4.     Age         int
5.     Name        string `gorm:"size:255"` // string默认长度为
    255，使用这种tag重设。
6.     Num         int    `gorm:"AUTO_INCREMENT"` // 自增
7.
8.     CreditCard  CreditCard // One-To-One (拥有一个 -
    CreditCard表的UserID作外键)
9.     Emails      []Email   // One-To-Many (拥有多个 -
    Email表的UserID作外键)
10.
11.    BillingAddress Address // One-To-One (属于 - 本表的
  
```

```

BillingAddressID作外键)
12.     BillingAddressID  sql.NullInt64
13.
14.     ShippingAddress  Address          // One-To-One (属于 - 本表的
ShippingAddressID作外键)
15.     ShippingAddressID int
16.
17.     IgnoreMe          int `gorm:"- "` // 忽略这个字段
18.     Languages         []Language `gorm:"many2many:user_languages;"`
// Many-To-Many , 'user_languages'是连接表
19. }
20.
21. type Email struct {
22.     ID      int
23.     UserID  int `gorm:"index"` // 外键 (属于), tag `index`是为该列
创建索引
24.     Email   string `gorm:"type:varchar(100);unique_index"` //
`type`设置sql类型, `unique_index` 为该列设置唯一索引
25.     Subscribed bool
26. }
27.
28. type Address struct {
29.     ID      int
30.     Address1 string `gorm:"not null;unique"` // 设置字段为非
空并唯一
31.     Address2 string `gorm:"type:varchar(100);unique"`
32.     Post     sql.NullString `gorm:"not null"`
33. }
34.
35. type Language struct {
36.     ID      int
37.     Name    string `gorm:"index:idx_name_code"` // 创建索引并命名, 如果找
到其他相同名称的索引则创建组合索引
38.     Code    string `gorm:"index:idx_name_code"` // `unique_index` also
works
39. }
40.
41. type CreditCard struct {

```

```

42.     gorm.Model
43.     UserID uint
44.     Number string
45. }

```

约定 {#c}

`gorm.Model` 结构体

基本模型定义 `gorm.Model`，包括字

段 `ID`，`CreatedAt`，`UpdatedAt`，`DeletedAt`，你可以将它嵌入你的模型，或者只写你想要的字段

```

1.  // 基本模型的定义
2.  type Model struct {
3.      ID          uint `gorm:"primary_key"`
4.      CreatedAt time.Time
5.      UpdatedAt time.Time
6.      DeletedAt *time.Time
7.  }
8.
9.  // 添加字段 `ID`，`CreatedAt`，`UpdatedAt`，`DeletedAt`
10. type User struct {
11.     gorm.Model
12.     Name string
13. }
14.
15. // 只需要字段 `ID`，`CreatedAt`
16. type User struct {
17.     ID          uint
18.     CreatedAt time.Time
19.     Name        string
20. }

```

表名是结构体名称的复数形式

```

1. type User struct {} // 默认表名是`users`
2.
3. // 设置User的表名为`profiles`
4. func (User) TableName() string {
5.     return "profiles"
6. }
7.
8. func (u User) TableName() string {
9.     if u.Role == "admin" {
10.         return "admin_users"
11.     } else {
12.         return "users"
13.     }
14. }
15.
16. // 全局禁用表名复数
17. db.SingularTable(true) // 如果设置为true, `User`的默认表名为`user`, 使用
    `TableName`设置的表名不受影响

```

更改默认表名

您可以通过定义 `DefaultTableNameHandler` 对默认表名应用任何规则。

```

1. gorm.DefaultTableNameHandler = func (db *gorm.DB, defaultTableName
    string) string {
2.     return "prefix_" + defaultTableName;
3. }

```

列名是字段名的蛇形小写

```

1. type User struct {
2.     ID uint           // 列名为 `id`
3.     Name string        // 列名为 `name`
4.     Birthday time.Time // 列名为 `birthday`
5.     CreatedAt time.Time // 列名为 `created_at`
6. }

```

```

7.
8. // 重设列名
9. type Animal struct {
10.     AnimalId    int64    `gorm:"column:beast_id"` // 设置列
        名为`beast_id`
11.     Birthday    time.Time `gorm:"column:day_of_the_beast"` // 设置列
        名为`day_of_the_beast`
12.     Age         int64    `gorm:"column:age_of_the_beast"` // 设置列
        名为`age_of_the_beast`
13. }

```

字段 `ID` 为主键

```

1. type User struct {
2.     ID    uint // 字段`ID`为默认主键
3.     Name string
4. }
5.
6. // 使用tag`primary_key`用来设置主键
7. type Animal struct {
8.     AnimalId int64 `gorm:"primary_key"` // 设置AnimalId为主键
9.     Name     string
10.    Age      int64
11. }

```

字段 `CreatedAt` 用于存储记录的创建时间

创建具有 `CreatedAt` 字段的记录将被设置为当前时间

```

1. db.Create(&user) // 将会设置`CreatedAt`为当前时间
2.
3. // 要更改它的值，你需要使用`Update`
4. db.Model(&user).Update("CreatedAt", time.Now())

```

字段 `UpdatedAt` 用于存储记录的修改时间

保存具有 `UpdatedAt` 字段的记录将被设置为当前时间

```
1. db.Save(&user) // 将会设置`UpdatedAt`为当前时间
2. db.Model(&user).Update("name", "jinzhu") // 将会设置`UpdatedAt`为当前时间
```

字段 `DeletedAt` 用于存储记录的删除时间，如果字段存在

删除具有 `DeletedAt` 字段的记录，它不会冲数据库中删除，但只将字段 `DeletedAt` 设置为当前时间，并在查询时无法找到记录，请参阅[软删除](#)

4. CRUD:读写数据

- **CRUD:读写数据**
 - **创建 {#c}**
 - 创建记录
 - 默认值
 - 在Callbacks中设置主键
 - 扩展创建选项
 - **查询 {#q}**
 - Where查询条件（简单SQL）
 - Where查询条件（Struct & Map）
 - Not条件查询
 - 带内联条件的查询
 - Or条件查询
 - 查询链
 - 扩展查询选项
 - FirstOrInit
 - Attrs
 - Assign
 - FirstOrCreate
 - Attrs
 - Assign
 - Select
 - Order
 - Limit
 - Offset
 - Count
 - Group & Having

- Join
- Pluck
- Scan {#Scan}
- Scopes {#Scopes}
- 指定表名
- 预加载 {#p}
 - 自定义预加载SQL
 - 嵌套预加载
- 更新 {#u}
 - 更新全部字段
 - 更新更改字段
 - 更新选择的字段
 - 更新更改字段但不进行Callbacks
 - Batch Updates 批量更新
 - 使用SQL表达式更新
 - 在Callbacks中更改更新值
 - 额外更新选项
- 删除/软删除 {#d}
 - 批量删除
 - 软删除
- 关联 {#a}
 - 创建/更新时跳过保存关联
 - tag设置跳过保存关联

CRUD:读写数据

创建 {#c}

创建记录

```

1. user := User{Name: "Jinzhu", Age: 18, Birthday: time.Now()}
2.
3. db.NewRecord(user) // => 主键为空返回`true`
4.
5. db.Create(&user)
6.
7. db.NewRecord(user) // => 创建`user`后返回`false`

```

默认值

您可以在gorm tag中定义默认值，然后插入SQL将忽略具有默认值的这些字段，并且其值为空，并且在将记录插入数据库后，gorm将从数据库加载这些字段的值。

```

1. type Animal struct {
2.     ID    int64
3.     Name  string `gorm:"default:'galeone'"`
4.     Age   int64
5. }
6.
7. var animal = Animal{Age: 99, Name: ""}
8. db.Create(&animal)
9. // INSERT INTO animals("age") values('99');
10. // SELECT name from animals WHERE ID=111; // 返回主键为 111
11. // animal.Name => 'galeone'

```

在Callbacks中设置主键

如果要在BeforeCreate回调中设置主字段的值，可以使用scope.SetColumn，例如：

```

1. func (user *User) BeforeCreate(scope *gorm.Scope) error {
2.     scope.SetColumn("ID", uuid.New())

```

```
3.     return nil
4. }
```

扩展创建选项

```
1. // 为Insert语句添加扩展SQL选项
2. db.Set("gorm:insert_option", "ON CONFLICT").Create(&product)
3. // INSERT INTO products (name, code) VALUES ("name", "code") ON
   CONFLICT;
```

查询 {#q}

```
1. // 获取第一条记录, 按主键排序
2. db.First(&user)
3. ///// SELECT * FROM users ORDER BY id LIMIT 1;
4.
5. // 获取最后一条记录, 按主键排序
6. db.Last(&user)
7. ///// SELECT * FROM users ORDER BY id DESC LIMIT 1;
8.
9. // 获取所有记录
10. db.Find(&users)
11. ///// SELECT * FROM users;
12.
13. // 使用主键获取记录
14. db.First(&user, 10)
15. ///// SELECT * FROM users WHERE id = 10;
```

Where查询条件 (简单SQL)

```
1. // 获取第一个匹配记录
2. db.Where("name = ?", "jinzhu").First(&user)
3. ///// SELECT * FROM users WHERE name = 'jinzhu' limit 1;
4.
5. // 获取所有匹配记录
```

```

6. db.Where("name = ?", "jinzhu").Find(&users)
7. ///// SELECT * FROM users WHERE name = 'jinzhu';
8.
9. db.Where("name <> ?", "jinzhu").Find(&users)
10.
11. // IN
12. db.Where("name in (?)", []string{"jinzhu", "jinzhu
    2"}).Find(&users)
13.
14. // LIKE
15. db.Where("name LIKE ?", "%jin%").Find(&users)
16.
17. // AND
18. db.Where("name = ? AND age >= ?", "jinzhu", "22").Find(&users)
19.
20. // Time
21. db.Where("updated_at > ?", lastWeek).Find(&users)
22.
23. db.Where("created_at BETWEEN ? AND ?", lastWeek,
    today).Find(&users)

```

Where查询条件 (Struct & Map)

注意：当使用struct查询时，GORM将只查询那些具有值的字段

```

1. // Struct
2. db.Where(&User{Name: "jinzhu", Age: 20}).First(&user)
3. ///// SELECT * FROM users WHERE name = "jinzhu" AND age = 20 LIMIT
    1;
4.
5. // Map
6. db.Where(map[string]interface{}{"name": "jinzhu", "age":
    20}).Find(&users)
7. ///// SELECT * FROM users WHERE name = "jinzhu" AND age = 20;
8.
9. // 主键的Slice
10. db.Where([]int64{20, 21, 22}).Find(&users)

```

```
11.  ///// SELECT * FROM users WHERE id IN (20, 21, 22);
```

Not条件查询

```
1.  db.Not("name", "jinzhu").First(&user)
2.  ///// SELECT * FROM users WHERE name <> "jinzhu" LIMIT 1;
3.
4.  // Not In
5.  db.Not("name", []string{"jinzhu", "jinzhu 2"}).Find(&users)
6.  ///// SELECT * FROM users WHERE name NOT IN ("jinzhu", "jinzhu 2");
7.
8.  // Not In slice of primary keys
9.  db.Not([]int64{1,2,3}).First(&user)
10.  ///// SELECT * FROM users WHERE id NOT IN (1,2,3);
11.
12.  db.Not([]int64{}).First(&user)
13.  ///// SELECT * FROM users;
14.
15.  // Plain SQL
16.  db.Not("name = ?", "jinzhu").First(&user)
17.  ///// SELECT * FROM users WHERE NOT(name = "jinzhu");
18.
19.  // Struct
20.  db.Not(User{Name: "jinzhu"}).First(&user)
21.  ///// SELECT * FROM users WHERE name <> "jinzhu";
```

带内联条件的查询

注意：使用主键查询时，应仔细检查所传递的值是否为有效主键，以避免SQL注入

```
1.  // 按主键获取
2.  db.First(&user, 23)
3.  ///// SELECT * FROM users WHERE id = 23 LIMIT 1;
4.
5.  // 简单SQL
```

```

6. db.Find(&user, "name = ?", "jinzhu")
7. ///// SELECT * FROM users WHERE name = "jinzhu";
8.
9. db.Find(&users, "name <> ? AND age > ?", "jinzhu", 20)
10. ///// SELECT * FROM users WHERE name <> "jinzhu" AND age > 20;
11.
12. // Struct
13. db.Find(&users, User{Age: 20})
14. ///// SELECT * FROM users WHERE age = 20;
15.
16. // Map
17. db.Find(&users, map[string]interface{}{"age": 20})
18. ///// SELECT * FROM users WHERE age = 20;

```

Or条件查询

```

1. db.Where("role = ?", "admin").Or("role = ?",
   "super_admin").Find(&users)
2. ///// SELECT * FROM users WHERE role = 'admin' OR role =
   'super_admin';
3.
4. // Struct
5. db.Where("name = 'jinzhu']").Or(User{Name: "jinzhu 2"}).Find(&users)
6. ///// SELECT * FROM users WHERE name = 'jinzhu' OR name = 'jinzhu
   2';
7.
8. // Map
9. db.Where("name = 'jinzhu']").Or(map[string]interface{}{"name":
   "jinzhu 2"}).Find(&users)

```

查询链

Gorm有一个可链接的API，你可以这样使用它

```

1. db.Where("name <> ?", "jinzhu").Where("age >= ? and role <>
   ?", 20, "admin").Find(&users)

```

```

2.  //// SELECT * FROM users WHERE name <> 'jinzhu' AND age >= 20 AND
    role <> 'admin';
3.
4.  db.Where("role = ?", "admin").Or("role = ?",
    "super_admin").Not("name = ?", "jinzhu").Find(&users)

```

扩展查询选项

```

1.  // 为Select语句添加扩展SQL选项
2.  db.Set("gorm:query_option", "FOR UPDATE").First(&user, 10)
3.  //// SELECT * FROM users WHERE id = 10 FOR UPDATE;

```

FirstOrInit

获取第一个匹配的记录，或者使用给定的条件初始化一个新的记录（仅适用于struct，map条件）

```

1.  // Unfound
2.  db.FirstOrInit(&user, User{Name: "non_existing"})
3.  //// user -> User{Name: "non_existing"}
4.
5.  // Found
6.  db.Where(User{Name: "Jinzhu"}).FirstOrInit(&user)
7.  //// user -> User{Id: 111, Name: "Jinzhu", Age: 20}
8.  db.FirstOrInit(&user, map[string]interface{}{"name": "jinzhu"})
9.  //// user -> User{Id: 111, Name: "Jinzhu", Age: 20}

```

Attrs

如果未找到记录，则使用参数初始化结构

```

1.  // Unfound
2.  db.Where(User{Name: "non_existing"}).Attrs(User{Age:
    20}).FirstOrInit(&user)
3.  //// SELECT * FROM USERS WHERE name = 'non_existing';

```

```

4.  //// user -> User{Name: "non_existing", Age: 20}
5.
6.  db.Where(User{Name: "non_existing"}).Attrs("age",
    20).FirstOrInit(&user)
7.  //// SELECT * FROM USERS WHERE name = 'non_existing';
8.  //// user -> User{Name: "non_existing", Age: 20}
9.
10. // Found
11. db.Where(User{Name: "Jinzhu"}).Attrs(User{Age:
    30}).FirstOrInit(&user)
12. //// SELECT * FROM USERS WHERE name = jinzhu';
13. //// user -> User{Id: 111, Name: "Jinzhu", Age: 20}

```

Assign

将参数分配给结果，不管它是否被找到

```

1. // Unfound
2. db.Where(User{Name: "non_existing"}).Assign(User{Age:
    20}).FirstOrInit(&user)
3. //// user -> User{Name: "non_existing", Age: 20}
4.
5. // Found
6. db.Where(User{Name: "Jinzhu"}).Assign(User{Age:
    30}).FirstOrInit(&user)
7. //// SELECT * FROM USERS WHERE name = jinzhu';
8. //// user -> User{Id: 111, Name: "Jinzhu", Age: 30}

```

FirstOrCreate

获取第一个匹配的记录，或创建一个具有给定条件的新记录（仅适用于 struct, map 条件）

```

1. // Unfound
2. db.FirstOrCreate(&user, User{Name: "non_existing"})
3. //// INSERT INTO "users" (name) VALUES ("non_existing");

```

```

4.  //// user -> User{Id: 112, Name: "non_existing"}
5.
6.  // Found
7.  db.Where(User{Name: "Jinzhu"}).FirstOrDefault(&user)
8.  //// user -> User{Id: 111, Name: "Jinzhu"}

```

Attrs

如果未找到记录，则为参数分配结构

```

1.  // Unfound
2.  db.Where(User{Name: "non_existing"}).Attrs(User{Age:
    20}).FirstOrDefault(&user)
3.  //// SELECT * FROM users WHERE name = 'non_existing';
4.  //// INSERT INTO "users" (name, age) VALUES ("non_existing", 20);
5.  //// user -> User{Id: 112, Name: "non_existing", Age: 20}
6.
7.  // Found
8.  db.Where(User{Name: "jinzhu"}).Attrs(User{Age:
    30}).FirstOrDefault(&user)
9.  //// SELECT * FROM users WHERE name = 'jinzhu';
10. //// user -> User{Id: 111, Name: "jinzhu", Age: 20}

```

Assign

将其分配给记录，而不管它是否被找到，并保存回数据库。

```

1.  // Unfound
2.  db.Where(User{Name: "non_existing"}).Assign(User{Age:
    20}).FirstOrDefault(&user)
3.  //// SELECT * FROM users WHERE name = 'non_existing';
4.  //// INSERT INTO "users" (name, age) VALUES ("non_existing", 20);
5.  //// user -> User{Id: 112, Name: "non_existing", Age: 20}
6.
7.  // Found
8.  db.Where(User{Name: "jinzhu"}).Assign(User{Age:

```

```

    30}).FirstOrCreate(&user)
9.  //// SELECT * FROM users WHERE name = 'jinzhu';
10.  //// UPDATE users SET age=30 WHERE id = 111;
11.  //// user -> User{Id: 111, Name: "jinzhu", Age: 30}

```

Select

指定要从数据库检索的字段，默认情况下，将选择所有字段；

```

1.  db.Select("name, age").Find(&users)
2.  //// SELECT name, age FROM users;
3.
4.  db.Select([]string{"name", "age"}).Find(&users)
5.  //// SELECT name, age FROM users;
6.
7.  db.Table("users").Select("COALESCE(age,?)", 42).Rows()
8.  //// SELECT COALESCE(age, '42') FROM users;

```

Order

在从数据库检索记录时指定顺序，将重排序设置为 `true` 以覆盖定义的条件

```

1.  db.Order("age desc, name").Find(&users)
2.  //// SELECT * FROM users ORDER BY age desc, name;
3.
4.  // Multiple orders
5.  db.Order("age desc").Order("name").Find(&users)
6.  //// SELECT * FROM users ORDER BY age desc, name;
7.
8.  // ReOrder
9.  db.Order("age desc").Find(&users1).Order("age", true).Find(&users2)
10.  //// SELECT * FROM users ORDER BY age desc; (users1)
11.  //// SELECT * FROM users ORDER BY age; (users2)

```

Limit

指定要检索的记录数

```

1. db.Limit(3).Find(&users)
2. ///// SELECT * FROM users LIMIT 3;
3.
4. // Cancel limit condition with -1
5. db.Limit(10).Find(&users1).Limit(-1).Find(&users2)
6. ///// SELECT * FROM users LIMIT 10; (users1)
7. ///// SELECT * FROM users; (users2)

```

Offset

指定在开始返回记录之前要跳过的记录数

```

1. db.Offset(3).Find(&users)
2. ///// SELECT * FROM users OFFSET 3;
3.
4. // Cancel offset condition with -1
5. db.Offset(10).Find(&users1).Offset(-1).Find(&users2)
6. ///// SELECT * FROM users OFFSET 10; (users1)
7. ///// SELECT * FROM users; (users2)

```

Count

获取模型的记录数

```

1. db.Where("name = ?", "jinzhu").Or("name = ?", "jinzhu
   2").Find(&users).Count(&count)
2. ///// SELECT * from USERS WHERE name = 'jinzhu' OR name = 'jinzhu
   2'; (users)
3. ///// SELECT count(*) FROM users WHERE name = 'jinzhu' OR name =
   'jinzhu 2'; (count)
4.
5. db.Model(&User{}).Where("name = ?", "jinzhu").Count(&count)

```

```

6.  //// SELECT count(*) FROM users WHERE name = 'jinzhu'; (count)
7.
8.  db.Table("deleted_users").Count(&count)
9.  //// SELECT count(*) FROM deleted_users;

```

Group & Having

```

1.  rows, err := db.Table("orders").Select("date(created_at) as date,
    sum(amount) as total").Group("date(created_at)").Rows()
2.  for rows.Next() {
3.      ...
4.  }
5.
6.  rows, err := db.Table("orders").Select("date(created_at) as date,
    sum(amount) as
    total").Group("date(created_at)").Having("sum(amount) > ?",
    100).Rows()
7.  for rows.Next() {
8.      ...
9.  }
10.
11. type Result struct {
12.     Date  time.Time
13.     Total int64
14. }
15. db.Table("orders").Select("date(created_at) as date, sum(amount) as
    total").Group("date(created_at)").Having("sum(amount) > ?",
    100).Scan(&results)

```

Join

指定连接条件

```

1.  rows, err := db.Table("users").Select("users.name,
    emails.email").Joins("left join emails on emails.user_id =
    users.id").Rows()

```

```

2. for rows.Next() {
3.     ...
4. }
5.
6. db.Table("users").Select("users.name, emails.email").Joins("left
   join emails on emails.user_id = users.id").Scan(&results)
7.
8. // 多个连接与参数
9. db.Joins("JOIN emails ON emails.user_id = users.id AND emails.email
   = ?", "jinzhu@example.org").Joins("JOIN credit_cards ON
   credit_cards.user_id = users.id").Where("credit_cards.number = ?",
   "411111111111").Find(&user)

```

Pluck

将模型中的单个列作为地图查询，如果要查询多个列，可以使用[Scan](#)

```

1. var ages []int64
2. db.Find(&users).Pluck("age", &ages)
3.
4. var names []string
5. db.Model(&User{}).Pluck("name", &names)
6.
7. db.Table("deleted_users").Pluck("name", &names)
8.
9. // 要返回多个列，做这样：
10. db.Select("name, age").Find(&users)

```

Scan {#Scan}

将结果扫描到另一个结构中。

```

1. type Result struct {
2.     Name string
3.     Age  int
4. }

```

```

5.
6. var result Result
7. db.Table("users").Select("name, age").Where("name = ?",
   3).Scan(&result)
8.
9. // Raw SQL
10. db.Raw("SELECT name, age FROM users WHERE name = ?",
   3).Scan(&result)

```

Scopes {#Scopes}

将当前数据库连接传递到 `func(*DB) *DB`，可以用于动态添加条件

```

1. func AmountGreaterThan1000(db *gorm.DB) *gorm.DB {
2.     return db.Where("amount > ?", 1000)
3. }
4.
5. func PaidWithCreditCard(db *gorm.DB) *gorm.DB {
6.     return db.Where("pay_mode_sign = ?", "C")
7. }
8.
9. func PaidWithCod(db *gorm.DB) *gorm.DB {
10.    return db.Where("pay_mode_sign = ?", "C")
11. }
12.
13. func OrderStatus(status []string) func (db *gorm.DB) *gorm.DB {
14.    return func (db *gorm.DB) *gorm.DB {
15.        return db.Scopes(AmountGreaterThan1000).Where("status in
   (?)", status)
16.    }
17. }
18.
19. db.Scopes(AmountGreaterThan1000, PaidWithCreditCard).Find(&orders)
20. // 查找所有信用卡订单和金额大于1000
21.
22. db.Scopes(AmountGreaterThan1000, PaidWithCod).Find(&orders)
23. // 查找所有COD订单和金额大于1000

```

```

24.
25. db.Scopes(OrderStatus([]string{"paid", "shipped"})).Find(&orders)
26. // 查找所有付费, 发货订单

```

指定表名

```

1. // 使用User结构定义创建`deleted_users`表
2. db.Table("deleted_users").CreateTable(&User{})
3.
4. var deleted_users []User
5. db.Table("deleted_users").Find(&deleted_users)
6. ///// SELECT * FROM deleted_users;
7.
8. db.Table("deleted_users").Where("name = ?", "jinzhu").Delete()
9. ///// DELETE FROM deleted_users WHERE name = 'jinzhu';

```

预加载 {#p}

```

1. db.Preload("Orders").Find(&users)
2. ///// SELECT * FROM users;
3. ///// SELECT * FROM orders WHERE user_id IN (1,2,3,4);
4.
5. db.Preload("Orders", "state NOT IN (?)", "cancelled").Find(&users)
6. ///// SELECT * FROM users;
7. ///// SELECT * FROM orders WHERE user_id IN (1,2,3,4) AND state NOT
  IN ('cancelled');
8.
9. db.Where("state = ?", "active").Preload("Orders", "state NOT IN
  (?)", "cancelled").Find(&users)
10. ///// SELECT * FROM users WHERE state = 'active';
11. ///// SELECT * FROM orders WHERE user_id IN (1,2) AND state NOT IN
  ('cancelled');
12.
13. db.Preload("Orders").Preload("Profile").Preload("Role").Find(&users)
14. ///// SELECT * FROM users;
15. ///// SELECT * FROM orders WHERE user_id IN (1,2,3,4); // has many

```

```
16.  //// SELECT * FROM profiles WHERE user_id IN (1,2,3,4); // has one
17.  //// SELECT * FROM roles WHERE id IN (4,5,6); // belongs to
```

自定义预加载SQL

您可以通过传递 `func(db *gorm.DB) *gorm.DB` （与[Scopes](#)的使用方法相同）来自定义预加载SQL，例如：

```
1.  db.Preload("Orders", func(db *gorm.DB) *gorm.DB {
2.      return db.Order("orders.amount DESC")
3.  }).Find(&users)
4.  //// SELECT * FROM users;
5.  //// SELECT * FROM orders WHERE user_id IN (1,2,3,4) order by
   orders.amount DESC;
```

嵌套预加载

```
1.  db.Preload("Orders.OrderItems").Find(&users)
2.  db.Preload("Orders", "state = ?",
   "paid").Preload("Orders.OrderItems").Find(&users)
```

更新 {#u}

更新全部字段

`Save` 将包括执行更新SQL时的所有字段，即使它没有更改

```
1.  db.First(&user)
2.
3.  user.Name = "jinzhu 2"
4.  user.Age = 100
5.  db.Save(&user)
6.
7.  //// UPDATE users SET name='jinzhu 2', age=100, birthday='2016-01-
```

```
01', updated_at = '2013-11-17 21:34:10' WHERE id=111;
```

更新更改字段

如果只想更新更改的字段，可以使用 `Update` , `Updates`

```
1. // 更新单个属性（如果更改）
2. db.Model(&user).Update("name", "hello")
3. ///// UPDATE users SET name='hello', updated_at='2013-11-17
   21:34:10' WHERE id=111;
4.
5. // 使用组合条件更新单个属性
6. db.Model(&user).Where("active = ?", true).Update("name", "hello")
7. ///// UPDATE users SET name='hello', updated_at='2013-11-17
   21:34:10' WHERE id=111 AND active=true;
8.
9. // 使用`map`更新多个属性，只会更新这些更改的字段
10. db.Model(&user).Updates(map[string]interface{}{"name": "hello",
    "age": 18, "actived": false})
11. ///// UPDATE users SET name='hello', age=18, actived=false,
    updated_at='2013-11-17 21:34:10' WHERE id=111;
12.
13. // 使用`struct`更新多个属性，只会更新这些更改的和非空白字段
14. db.Model(&user).Updates(User{Name: "hello", Age: 18})
15. ///// UPDATE users SET name='hello', age=18, updated_at = '2013-11-
    17 21:34:10' WHERE id = 111;
16.
17. // 警告:当使用struct更新时，FORM将仅更新具有非空值的字段
18. // 对于下面的更新，什么都不会更新为"", 0, false是其类型的空白值
19. db.Model(&user).Updates(User{Name: "", Age: 0, Actived: false})
```

更新选择的字段

如果您只想在更新时更新或忽略某些字段，可以使用 `Select` , `Omit`

```
1. db.Model(&user).Select("name").Updates(map[string]interface{})
```

```

    {"name": "hello", "age": 18, "actived": false})
2.  //// UPDATE users SET name='hello', updated_at='2013-11-17
    21:34:10' WHERE id=111;
3.
4.  db.Model(&user).Omit("name").Updates(map[string]interface{}{"name":
    "hello", "age": 18, "actived": false})
5.  //// UPDATE users SET age=18, actived=false, updated_at='2013-11-17
    21:34:10' WHERE id=111;

```

更新更改字段但不进行Callbacks

以上更新操作将执行模型的 `BeforeUpdate` , `AfterUpdate` 方法, 更新其 `UpdatedAt` 时间戳, 在更新时保存它的 `Associations` , 如果不想调用它们, 可以使用 `UpdateColumn` , `UpdateColumns`

```

1.  // 更新单个属性, 类似于`Update`
2.  db.Model(&user).UpdateColumn("name", "hello")
3.  //// UPDATE users SET name='hello' WHERE id = 111;
4.
5.  // 更新多个属性, 与“更新”类似
6.  db.Model(&user).UpdateColumns(User{Name: "hello", Age: 18})
7.  //// UPDATE users SET name='hello', age=18 WHERE id = 111;

```

Batch Updates 批量更新

`Callbacks` 在批量更新时不会运行

```

1.  db.Table("users").Where("id IN (?)", []int{10,
    11}).Updates(map[string]interface{}{"name": "hello", "age": 18})
2.  //// UPDATE users SET name='hello', age=18 WHERE id IN (10, 11);
3.
4.  // 使用struct更新仅适用于非零值, 或使用map[string]interface{}
5.  db.Model(User{}).Updates(User{Name: "hello", Age: 18})
6.  //// UPDATE users SET name='hello', age=18;
7.
8.  // 使用`RowsAffected`获取更新记录计数

```

```
9. db.Model(User{}).Updates(User{Name: "hello", Age: 18}).RowsAffected
```

使用SQL表达式更新

```
1. DB.Model(&product).Update("price", gorm.Expr("price * ? + ?", 2,
    100))
2. ///// UPDATE "products" SET "price" = price * '2' + '100',
    "updated_at" = '2013-11-17 21:34:10' WHERE "id" = '2';
3.
4. DB.Model(&product).Updates(map[string]interface{}{"price":
    gorm.Expr("price * ? + ?", 2, 100)})
5. ///// UPDATE "products" SET "price" = price * '2' + '100',
    "updated_at" = '2013-11-17 21:34:10' WHERE "id" = '2';
6.
7. DB.Model(&product).UpdateColumn("quantity", gorm.Expr("quantity -
    ?", 1))
8. ///// UPDATE "products" SET "quantity" = quantity - 1 WHERE "id" =
    '2';
9.
10. DB.Model(&product).Where("quantity > 1").UpdateColumn("quantity",
    gorm.Expr("quantity - ?", 1))
11. ///// UPDATE "products" SET "quantity" = quantity - 1 WHERE "id" =
    '2' AND quantity > 1;
```

在Callbacks中更改更新值

如果要使用 `BeforeUpdate` , `BeforeSave` 更改回调中的更新值, 可以使用 `scope.SetColumn` , 例如

```
1. func (user *User) BeforeSave(scope *gorm.Scope) (err error) {
2.     if pw, err := bcrypt.GenerateFromPassword(user.Password, 0); err
        == nil {
3.         scope.SetColumn("EncryptedPassword", pw)
4.     }
5. }
```

额外更新选项

```
1. // 为Update语句添加额外的SQL选项
2. db.Model(&user).Set("gorm:update_option", "OPTION (OPTIMIZE FOR
   UNKNOWN)").Update("name, 'hello'")
3. ///// UPDATE users SET name='hello', updated_at = '2013-11-17
   21:34:10' WHERE id=111 OPTION (OPTIMIZE FOR UNKNOWN);
```

删除/软删除 {#d}

警告 删除记录时，需要确保其主要字段具有值，GORM将使用主键删除记录，如果主要字段为空，GORM将删除模型的所有记录

```
1. // 删除存在的记录
2. db.Delete(&email)
3. ///// DELETE from emails where id=10;
4.
5. // 为Delete语句添加额外的SQL选项
6. db.Set("gorm:delete_option", "OPTION (OPTIMIZE FOR
   UNKNOWN)").Delete(&email)
7. ///// DELETE from emails where id=10 OPTION (OPTIMIZE FOR UNKNOWN);
```

批量删除

删除所有匹配记录

```
1. db.Where("email LIKE ?", "%jinzhu%").Delete(Email{})
2. ///// DELETE from emails where email LIKE "%jinhu%";
3.
4. db.Delete(Email{}, "email LIKE ?", "%jinzhu%")
5. ///// DELETE from emails where email LIKE "%jinhu%";
```

软删除

如果模型有 `DeletedAt` 字段，它将自动获得软删除功能！那么在调用 `Delete` 时不会从数据库中永久删除，而是只将字段 `DeletedAt` 的值设置为当前时间。

```

1. db.Delete(&user)
2. ///// UPDATE users SET deleted_at="2013-10-29 10:23" WHERE id = 111;
3.
4. // 批量删除
5. db.Where("age = ?", 20).Delete(&User{})
6. ///// UPDATE users SET deleted_at="2013-10-29 10:23" WHERE age = 20;
7.
8. // 软删除的记录将在查询时被忽略
9. db.Where("age = 20").Find(&user)
10. ///// SELECT * FROM users WHERE age = 20 AND deleted_at IS NULL;
11.
12. // 使用Unscoped查找软删除的记录
13. db.Unscoped().Where("age = 20").Find(&users)
14. ///// SELECT * FROM users WHERE age = 20;
15.
16. // 使用Unscoped永久删除记录
17. db.Unscoped().Delete(&order)
18. ///// DELETE FROM orders WHERE id=10;

```

关联 {#a}

默认情况下，当创建/更新记录时，GORM将保存其关联，如果关联具有主键，GORM将调用Update来保存它，否则将被创建。

```

1. user := User{
2.     Name: "jinzhu",
3.     BillingAddress: Address{Address1: "Billing Address - Address
4.         1"},
5.     ShippingAddress: Address{Address1: "Shipping Address - Address
6.         1"},
7.     Emails: []Email{

```

```

6.                                     {Email:
    "jinzhu@example.com"},
7.                                     {Email: "jinzhu-
    2@example@example.com"},
8.                                     },
9.     Languages:      []Language{
10.                     {Name: "ZH"},
11.                     {Name: "EN"},
12.                     },
13. }
14.
15. db.Create(&user)
16. //// BEGIN TRANSACTION;
17. //// INSERT INTO "addresses" (address1) VALUES ("Billing Address -
    Address 1");
18. //// INSERT INTO "addresses" (address1) VALUES ("Shipping Address -
    Address 1");
19. //// INSERT INTO "users"
    (name,billing_address_id,shipping_address_id) VALUES ("jinzhu", 1,
    2);
20. //// INSERT INTO "emails" (user_id,email) VALUES (111,
    "jinzhu@example.com");
21. //// INSERT INTO "emails" (user_id,email) VALUES (111, "jinzhu-
    2@example.com");
22. //// INSERT INTO "languages" ("name") VALUES ('ZH');
23. //// INSERT INTO user_languages ("user_id","language_id") VALUES
    (111, 1);
24. //// INSERT INTO "languages" ("name") VALUES ('EN');
25. //// INSERT INTO user_languages ("user_id","language_id") VALUES
    (111, 2);
26. //// COMMIT;
27.
28. db.Save(&user)

```

参考[Associations](#)更多详细信息

创建/更新时跳过保存关联

默认情况下保存记录时，GORM也会保存它的关联，你可以通过设置 `gorm:save_associations` 为 `false` 跳过它。

```
1. db.Set("gorm:save_associations", false).Create(&user)
2.
3. db.Set("gorm:save_associations", false).Save(&user)
```

tag设置跳过保存关联

您可以使用tag来配置您的struct，以便在创建/更新时不会保存关联

```
1. type User struct {
2.     gorm.Model
3.     Name      string
4.     CompanyID uint
5.     Company   Company `gorm:"save_associations:false"`
6. }
7.
8. type Company struct {
9.     gorm.Model
10.    Name string
11. }
```

5. 关联

- 关联
 - 属于 {#bt}
 - 包含一个 {#ho}
 - 包含多个 {#hm}
 - 多对多 {#mtm}
 - 多种包含 {#p}
 - 关联模式 {#am}

关联

属于 {#bt}

```

1. // `User`属于`Profile`, `ProfileID`为外键
2. type User struct {
3.     gorm.Model
4.     Profile Profile
5.     ProfileID int
6. }
7.
8. type Profile struct {
9.     gorm.Model
10.    Name string
11. }
12.
13. db.Model(&user).Related(&profile)
14. ///// SELECT * FROM profiles WHERE id = 111; // 111是user的外键
    ProfileID

```

指定外键

```

1. type Profile struct {

```

```

2.     gorm.Model
3.     Name string
4. }
5.
6. type User struct {
7.     gorm.Model
8.     Profile      Profile `gorm:"ForeignKey:ProfileRefer"` // 使用
        ProfileRefer作为外键
9.     ProfileRefer int
10. }

```

指定外键和关联外键

```

1. type Profile struct {
2.     gorm.Model
3.     Refer string
4.     Name string
5. }
6.
7. type User struct {
8.     gorm.Model
9.     Profile Profile
        `gorm:"ForeignKey:ProfileID;AssociationForeignKey:Refer"`
10.    ProfileID int
11. }

```

包含一个 {#ho}

```

1. // User 包含一个 CreditCard, UserID 为外键
2. type User struct {
3.     gorm.Model
4.     CreditCard CreditCard
5. }
6.
7. type CreditCard struct {
8.     gorm.Model

```

```

9.     UserID    uint
10.     Number    string
11. }
12.
13. var card CreditCard
14. db.Model(&user).Related(&card, "CreditCard")
15. ///// SELECT * FROM credit_cards WHERE user_id = 123; // 123 is
    user's primary key
16. // CreditCard是user的字段名称，这意味着获得user的CreditCard关系并将其填充
    到变量
17. // 如果字段名与变量的类型名相同，如上例所示，可以省略，如：
18. db.Model(&user).Related(&card)

```

指定外键

```

1. type Profile struct {
2.     gorm.Model
3.     Name      string
4.     UserRefer uint
5. }
6.
7. type User struct {
8.     gorm.Model
9.     Profile Profile `gorm:"ForeignKey:UserRefer"`
10. }

```

指定外键和关联外键

```

1. type Profile struct {
2.     gorm.Model
3.     Name      string
4.     UserID    uint
5. }
6.
7. type User struct {
8.     gorm.Model
9.     Refer      string

```

```

10.     Profile Profile
        `gorm:"ForeignKey:UserID;AssociationForeignKey:Refer"`
11. }

```

包含多个 {#hm}

```

1. // User 包含多个 emails, UserID 为外键
2. type User struct {
3.     gorm.Model
4.     Emails []Email
5. }
6.
7. type Email struct {
8.     gorm.Model
9.     Email string
10.    UserID uint
11. }
12.
13. db.Model(&user).Related(&emails)
14. ///// SELECT * FROM emails WHERE user_id = 111; // 111 是 user 的主键

```

指定外键

```

1. type Profile struct {
2.     gorm.Model
3.     Name string
4.     UserRefer uint
5. }
6.
7. type User struct {
8.     gorm.Model
9.     Profiles []Profile `gorm:"ForeignKey:UserRefer"`
10. }

```

指定外键和关联外键

```

1. type Profile struct {
2.     gorm.Model
3.     Name string
4.     UserID uint
5. }
6.
7. type User struct {
8.     gorm.Model
9.     Refer string
10.    Profiles []Profile
11.    `gorm:"ForeignKey:UserID;AssociationForeignKey:Refer"`
12. }

```

多对多 {#mtm}

```

1. // User 包含并属于多个 languages, 使用 `user_languages` 表连接
2. type User struct {
3.     gorm.Model
4.     Languages []Language `gorm:"many2many:user_languages;"`
5. }
6.
7. type Language struct {
8.     gorm.Model
9.     Name string
10. }
11.
12. db.Model(&user).Related(&languages, "Languages")
13. ///// SELECT * FROM "languages" INNER JOIN "user_languages" ON
14.    "user_languages"."language_id" = "languages"."id" WHERE
15.    "user_languages"."user_id" = 111

```

指定外键和关联外键

```

1. type CustomizePerson struct {
2.     IdPerson string `gorm:"primary_key:true"`
3.     Accounts []CustomizeAccount

```

```

    `gorm:"many2many:PersonAccount;ForeignKey:IdPerson;AssociationForeign
4. }
5.
6. type CustomizeAccount struct {
7.     IdAccount string `gorm:"primary_key:true"`
8.     Name      string
9. }

```

译者注：这里设置好像缺失一部分

多种包含 {#p}

支持多种的包含一个和包含多个的关联

```

1. type Cat struct {
2.     Id    int
3.     Name  string
4.     Toy   Toy `gorm:"polymorphic:Owner;"`
5. }
6.
7. type Dog struct {
8.     Id    int
9.     Name  string
10.    Toy   Toy `gorm:"polymorphic:Owner;"`
11. }
12.
13. type Toy struct {
14.     Id        int
15.     Name      string
16.     OwnerId   int
17.     OwnerType string
18. }

```

注意：多态属性和多对多显式不支持，并且会抛出错误。

关联模式 {#am}

关联模式包含一些帮助方法来处理关系事情很容易。

```

1. // 开始关联模式
2. var user User
3. db.Model(&user).Association("Languages")
4. // `user`是源，它需要是一个有效的记录（包含主键）
5. // `Languages`是关系中源的字段名。
6. // 如果这些条件不匹配，将返回一个错误，检查它：
7. // db.Model(&user).Association("Languages").Error
8.
9.
10. // Query - 查找所有相关关联
11. db.Model(&user).Association("Languages").Find(&languages)
12.
13.
14. // Append - 添加新的many2many, has_many关联，会替换掉当前 has_one,
    belongs_to关联
15. db.Model(&user).Association("Languages").Append([]Language{languageZH,
    languageEN})
16. db.Model(&user).Association("Languages").Append(Language{Name:
    "DE"})
17.
18.
19. // Delete - 删除源和传递的参数之间的关系，不会删除这些参数
20. db.Model(&user).Association("Languages").Delete([]Language{languageZH,
    languageEN})
21. db.Model(&user).Association("Languages").Delete(languageZH,
    languageEN)
22.
23.
24. // Replace - 使用新的关联替换当前关联
25. db.Model(&user).Association("Languages").Replace([]Language{languageZH,
    languageEN})
26. db.Model(&user).Association("Languages").Replace(Language{Name:
    "DE"}, languageEN)
27.

```

```
28.  
29. // Count - 返回当前关联的计数  
30. db.Model(&user).Association("Languages").Count()  
31.  
32.  
33. // Clear - 删除源和当前关联之间的关系，不会删除这些关联  
34. db.Model(&user).Association("Languages").Clear()
```

6. Callbacks

- Callbacks
 - 创建对象
 - 更新对象
 - 删除对象
 - 查询对象 {#querying-an-object}
 - 回调示例

Callbacks

您可以将回调方法定义为模型结构的指针，在创建，更新，查询，删除时将被调用，如果任何回调返回错误，gorm将停止未来操作并回滚所有更改。

创建对象

创建过程中可用的回调

```

1. // begin transaction 开始事物
2. BeforeSave
3. BeforeCreate
4. // save before associations 保存前关联
5. // update timestamp `CreatedAt`, `UpdatedAt` 更新`CreatedAt`,
   `UpdatedAt`时间戳
6. // save self 保存自己
7. // reload fields that have default value and its value is blank 重新
   加载具有默认值且其值为空的字段
8. // save after associations 保存后关联
9. AfterCreate
10. AfterSave
11. // commit or rollback transaction 提交或回滚事务

```

更新对象

更新过程中可用的回调

1. `// begin transaction` 开始事物
2. `BeforeSave`
3. `BeforeUpdate`
4. `// save before associations` 保存前关联
5. `// update timestamp `UpdatedAt`` 更新`UpdatedAt`时间戳
6. `// save self` 保存自己
7. `// save after associations` 保存后关联
8. `AfterUpdate`
9. `AfterSave`
10. `// commit or rollback transaction` 提交或回滚事务

删除对象

删除过程中可用的回调

1. `// begin transaction` 开始事物
2. `BeforeDelete`
3. `// delete self` 删除自己
4. `AfterDelete`
5. `// commit or rollback transaction` 提交或回滚事务

查询对象 {#querying-an-object}

查询过程中可用的回调

1. `// load data from database` 从数据库加载数据
2. `// Preloading (eager loading)` 预加载（加载）
3. `AfterFind`

回调示例

```

1. func (u *User) BeforeUpdate() (err error) {
2.     if u.readonly() {
3.         err = errors.New("read only user")
4.     }
5.     return
6. }
7.
8. // 如果用户ID大于1000, 则回滚插入
9. func (u *User) AfterCreate() (err error) {
10.    if (u.Id > 1000) {
11.        err = errors.New("user id is already greater than 1000")
12.    }
13.    return
14. }

```

gorm中的保存/删除操作正在事务中运行，因此在该事务中所做的更改不可见，除非提交。 如果要在回调中使用这些更改，则需要在同一事务中运行SQL。 所以你需要传递当前事务到回调，像这样：

```

1. func (u *User) AfterCreate(tx *gorm.DB) (err error) {
2.     tx.Model(u).Update("role", "admin")
3.     return
4. }

```

```

1. func (u *User) AfterCreate(scope *gorm.Scope) (err error) {
2.     scope.DB().Model(u).Update("role", "admin")
3.     return
4. }

```

7. 高级用法

- 高级用法
 - 错误处理 {#eh}
 - 事物 {#t}
 - 一个具体的例子
 - SQL构建 {#sb}
 - 执行原生SQL
 - sql.Row & sql.Rows
 - 迭代中使用sql.Rows的Scan
 - 通用数据库接口sql.DB {#g}
 - 连接池
 - 复合主键 {#cpk}
 - 日志 {#l}
 - 自定义日志

高级用法

错误处理 {#eh}

执行任何操作后，如果发生任何错误，GORM将其设置为 `*DB` 的 `Error` 字段

```

1. if err := db.Where("name = ?", "jinzhu").First(&user).Error; err !=
   nil {
2.     // 错误处理...
3. }
4.
5. // 如果有多个错误发生，用`GetErrors`获取所有的错误，它返回`[]error`
6. db.First(&user).Limit(10).Find(&users).GetErrors()
7.

```

```

8. // 检查是否返回RecordNotFound错误
9. db.Where("name = ?", "hello world").First(&user).RecordNotFound()
10.
11. if db.Model(&user).Related(&credit_card).RecordNotFound() {
12.     // 没有信用卡被发现处理...
13. }

```

事物 {#t}

要在事务中执行一组操作，一般流程如下。

```

1. // 开始事务
2. tx := db.Begin()
3.
4. // 在事务中做一些数据库操作（从这一点使用'tx'，而不是'db'）
5. tx.Create(...)
6.
7. // ...
8.
9. // 发生错误时回滚事务
10. tx.Rollback()
11.
12. // 或提交事务
13. tx.Commit()

```

一个具体的例子

```

1. func CreateAnimals(db *gorm.DB) err {
2.     tx := db.Begin()
3.     // 注意，一旦你在一个事务中，使用tx作为数据库句柄
4.
5.     if err := tx.Create(&Animal{Name: "Giraffe"}).Error; err != nil {
6.         tx.Rollback()
7.         return err
8.     }
9.

```

```

10.     if err := tx.Create(&Animal{Name: "Lion"}).Error; err != nil {
11.         tx.Rollback()
12.         return err
13.     }
14.
15.     tx.Commit()
16.     return nil
17. }

```

SQL构建 {#sb}

执行原生SQL

```

1.  db.Exec("DROP TABLE users;")
2.  db.Exec("UPDATE orders SET shipped_at=? WHERE id IN (?)", time.Now,
    []int64{11,22,33})
3.
4.  // Scan
5.  type Result struct {
6.      Name string
7.      Age  int
8.  }
9.
10. var result Result
11. db.Raw("SELECT name, age FROM users WHERE name = ?",
    3).Scan(&result)

```

sql.Row & sql.Rows

获取查询结果为 `*sql.Row` 或 `*sql.Rows`

```

1.  row := db.Table("users").Where("name = ?", "jinzhu").Select("name,
    age").Row() // (*sql.Row)
2.  row.Scan(&name, &age)
3.

```

```

4. rows, err := db.Model(&User{}).Where("name = ?",
    "jinzhu").Select("name, age, email").Rows() // (*sql.Rows, error)
5. defer rows.Close()
6. for rows.Next() {
7.     ...
8.     rows.Scan(&name, &age, &email)
9.     ...
10. }
11.
12. // Raw SQL
13. rows, err := db.Raw("select name, age, email from users where name
    = ?", "jinzhu").Rows() // (*sql.Rows, error)
14. defer rows.Close()
15. for rows.Next() {
16.     ...
17.     rows.Scan(&name, &age, &email)
18.     ...
19. }

```

迭代中使用sql.Rows的Scan

```

1. rows, err := db.Model(&User{}).Where("name = ?",
    "jinzhu").Select("name, age, email").Rows() // (*sql.Rows, error)
2. defer rows.Close()
3.
4. for rows.Next() {
5.     var user User
6.     db.ScanRows(rows, &user)
7.     // do something
8. }

```

通用数据库接口sql.DB {#g}

从 `*gorm.DB` 连接获取通用数据库接口 `*sql.DB`

```

1. // 获取通用数据库对象`*sql.DB`以使用其函数

```

```

2. db.DB()
3.
4. // Ping
5. db.DB().Ping()

```

连接池

```

1. db.DB().SetMaxIdleConns(10)
2. db.DB().SetMaxOpenConns(100)

```

复合主键 {#cpk}

将多个字段设置为主键以启用复合主键

```

1. type Product struct {
2.     ID          string `gorm:"primary_key"`
3.     LanguageCode string `gorm:"primary_key"`
4. }

```

日志 {#l}

Gorm有内置的日志记录器支持，默认情况下，它会打印发生的错误

```

1. // 启用Logger, 显示详细日志
2. db.LogMode(true)
3.
4. // 禁用日志记录器, 不显示任何日志
5. db.LogMode(false)
6.
7. // 调试单个操作, 显示此操作的详细日志
8. db.Debug().Where("name = ?", "jinzhu").First(&User{})

```

自定义日志

参考GORM的默认记录器如何自定义它

<https://github.com/jinzhu/gorm/blob/master/logger.go>

```
1. db.SetLogger(gorm.Logger{revel.TRACE})
2. db.SetLogger(log.New(os.Stdout, "\r\n", 0))
```

8. 开发

- 开发
 - 架构 {#a}
 - 写插件 {#w}
 - 注册新的callback
 - 删除现有的callback
 - 替换现有的callback
 - 注册callback顺序
 - 预定义回调

开发

架构 {#a}

Gorm使用可链接的API，`*gorm.DB` 是链的桥梁，对于每个链API，它将创建一个新的关系。

```
1. db, err := gorm.Open("postgres", "user=gorm dbname=gorm
   sslmode=disable")
2.
3. // 创建新关系
4. db = db.Where("name = ?", "jinzhu")
5.
6. // 过滤更多
7. if SomeCondition {
8.     db = db.Where("age = ?", 20)
9. } else {
10.    db = db.Where("age = ?", 30)
11. }
12. if YetAnotherCondition {
13.    db = db.Where("active = ?", 1)
14. }
```

当我们开始执行任何操作时，GORM将基于当前的 `*gorm.DB` 创建一个新的 `*gorm.Scope` 实例

```
1. // 执行查询操作
2. db.First(&user)
```

并且基于当前操作的类型，它将调用注册的 `creating` , `updating` , `querying` , `deleting` 或 `row_querying` 回调来运行操作。

对于上面的例子，将调用 `querying` , 参考[查询回调](#)

写插件 {#w}

GORM本身由 `Callbacks` 提供支持，因此您可以根据需要完全自定义 GORM

注册新的callback

```
1. func updateCreated(scope *Scope) {
2.     if scope.HasColumn("Created") {
3.         scope.SetColumn("Created", NowFunc())
4.     }
5. }
6.
7. db.Callback().Create().Register("update_created_at", updateCreated)
8. // 注册Create进程的回调
```

删除现有的callback

```
1. db.Callback().Create().Remove("gorm:create")
2. // 从Create回调中删除`gorm:create`回调
```

替换现有的callback

```
1. db.Callback().Create().Replace("gorm:create", newCreateFunction)
2. // 使用新函数`newCreateFunction`替换回调`gorm:create`用于创建过程
```

注册callback顺序

```
1. db.Callback().Create().Before("gorm:create").Register("update_created", updateCreated)
2. db.Callback().Create().After("gorm:create").Register("update_created", updateCreated)
3. db.Callback().Query().After("gorm:query").Register("my_plugin:after_query", afterQuery)
4. db.Callback().Delete().After("gorm:delete").Register("my_plugin:after_delete", afterDelete)
5. db.Callback().Update().Before("gorm:update").Register("my_plugin:before_update", beforeUpdate)
6. db.Callback().Create().Before("gorm:create").After("gorm:before_create", beforeCreate)
```

预定义回调

GORM定义了回调以执行其CRUD操作，在开始编写插件之前检查它们。

- [Create callbacks](#)
- [Update callbacks](#)
- [Query callbacks](#)
- [Delete callbacks](#)
- Row Query callbacks

Row Query callbacks将在运行 `Row` 或 `Rows` 时被调用，默认情况下没有注册的回调，你可以注册一个新的回调：

```
1. func updateTableName(scope *gorm.Scope) {
```

```
2.    scope.Search.Table(scope.TableName() + "_draft") // append
      `_draft` to table name
3.  }
4.
5.  db.Callback().RowQuery().Register("publish:update_table_name",
    updateTableName)
```

9. 更新日志

- 更新日志
 - v1.0
 - 破坏性变更

更新日志

v1.0

破坏性变更

- `gorm.Open` 返回类型为 `*gorm.DB` 而不是 `gorm.DB`
- 更新只会更新更改的字段

大多数应用程序不会受到影响，只有当您更改回调中的更新值

（如 `BeforeSave`，`BeforeUpdate`）时，应该使用 `scope.SetColumn`，
例如：

```
1. func (user *User) BeforeUpdate(scope *gorm.Scope) {
2.     if pw, err := bcrypt.GenerateFromPassword(user.Password, 0); err
   == nil {
3.         scope.SetColumn("EncryptedPassword", pw)
4.         // user.EncryptedPassword = pw // 不工作，更新时不会包括
   EncryptedPassword字段
5.     }
6. }
```

- 软删除的默认查询作用域只会检查 `deleted_at IS NULL`

之前它会检查`deleted_at`小于`0001-01-02`也排除空白时间，如：

```
1. SELECT * FROM users WHERE deleted_at IS NULL OR deleted_at <=
```

```
'0001-01-02'
```

但是没有必要，如果你使用 `*time.Time` 作为模型的 `DeletedAt`，它已经被 `gorm.Model` 使用了，所以SQL就足够了

```
1. SELECT * FROM users WHERE deleted_at IS NULL
```

所以如果你使用 `gorm.Model`，那么你是好的，没有什么需要改变，只要确保所有记录的空白时间为 `deleted_at` 设置为 `NULL`，示例迁移脚本：

```
1. import (
2.     "github.com/jinzhu/now"
3. )
4.
5. func main() {
6.     var models = []interface{}{&User{}, &Image{}}
7.     for _, model := range models {
8.         db.Unscoped().Model(model).Where("deleted_at < ?",
            now.MustParse("0001-01-02")).Update("deleted_at",
            gorm.Expr("NULL"))
9.     }
10. }
```

- 新的ToDBName逻辑

在GORM将struct, Field的名称转换为db名称之前，只有那些来自 `golint` 的常见初始化（如 `HTTP`，`URI`）是特殊处理的。

所以字段 `HTTP` 的数据库名称将是 `http` 而不是 `h_t_t_p`，但是一些其他的初始化，如 `SKU` 不在 `golint`，它的数据库名称将是 `s_k_u`，这看起来很丑陋，这个版本固定这个，任何大写的初始化应该正确转换。

- 错误 `RecordNotFound` 已重命名为 `ErrRecordNotFound`
- `mssql` 驱动程序已从默认驱动程序中删除，导入它用 `import _ "github.com/jinzhu/gorm/dialects/mssql"`
- `Hstore` 已移至 `github.com/jinzhu/gorm/dialects/postgres`

10. 译者日志

- [译者日志](#)

译者日志

翻译于： 20170111

有问题请邮件： sorex@163.com

本人blog： <http://sorex.cnblogs.com/>